操作系统原理 lab4 实验报告

实验课程:操作系统原理

实验名称:**中断**

专业名称:计算机科学与技术

学生姓名:梁力航

学生学号:23336128

实验地点:东实验楼 B201

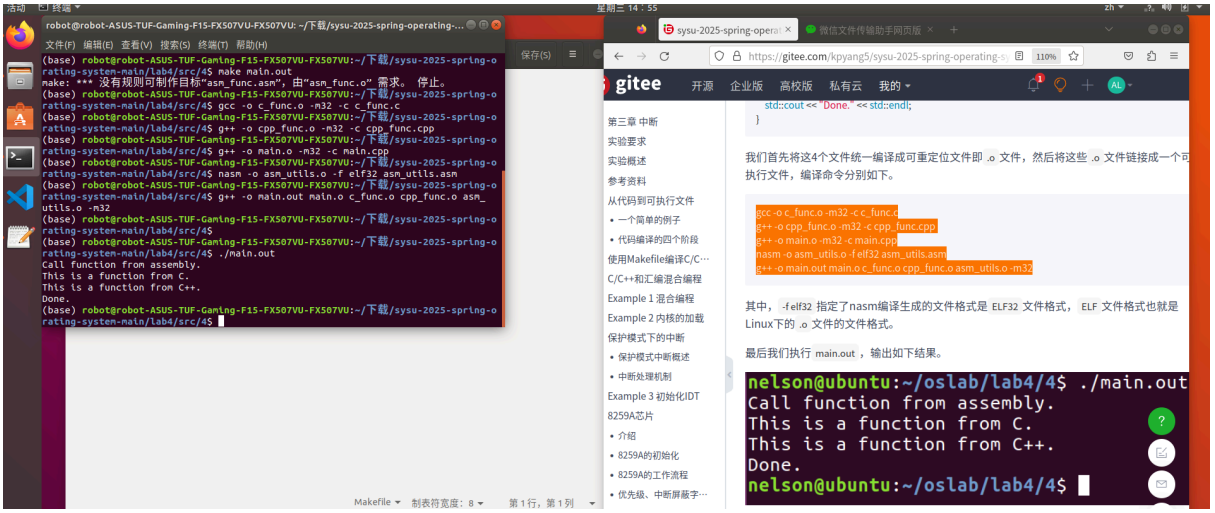
1. 实验要求

1. 阅读实验教程，掌握基础知识
2. 复现Example 1，理解**混合编程基本思路**
3. **用C/C++编写内核**
4. 复现Example 3，实现自己编写的**中断处理**
5. 复现Example 4，实现自己编写的**时钟中断处理**

2. 实验过程

Assignment 1 混合编程的基本思路

复现Example1



C/C++与汇编混合编程说明

关键字作用解析

1. `**global**` **关键字**：

- 位于汇编文件中，将标识符声明为全局可见

- 使标识符可被链接器识别，允许其他文件引用该标识符

- 示例：`global function_from_asm` 使该函数可被C/C++代码调用

2. `**extern**` **关键字**：

- 在C/C++中，声明函数或变量定义在其他文件中

- 在汇编中，表示引用外部函数

- 示例：`extern void function_from_asm();`表示函数定义在其他地方

3. `**extern "C"**`：

- 用于C++代码中，告诉编译器按C语言的命名规则处理函数

- 防止C++名称修饰(name mangling)机制导致链接错误

- 因为C++支持函数重载，编译后的函数名会被修饰，而C语言不会

- 示例：`extern "C" void function_from_CPP()`使此函数可被汇编代码按C函数方式调用

函数调用规则

混合编程中的函数调用规则：

- 参数从右向左依次入栈
- 返回值存放在eax寄存器中

- 基于ebp的寻址方式访问参数和局部变量

Make构建说明

Makefile的关键组成部分：

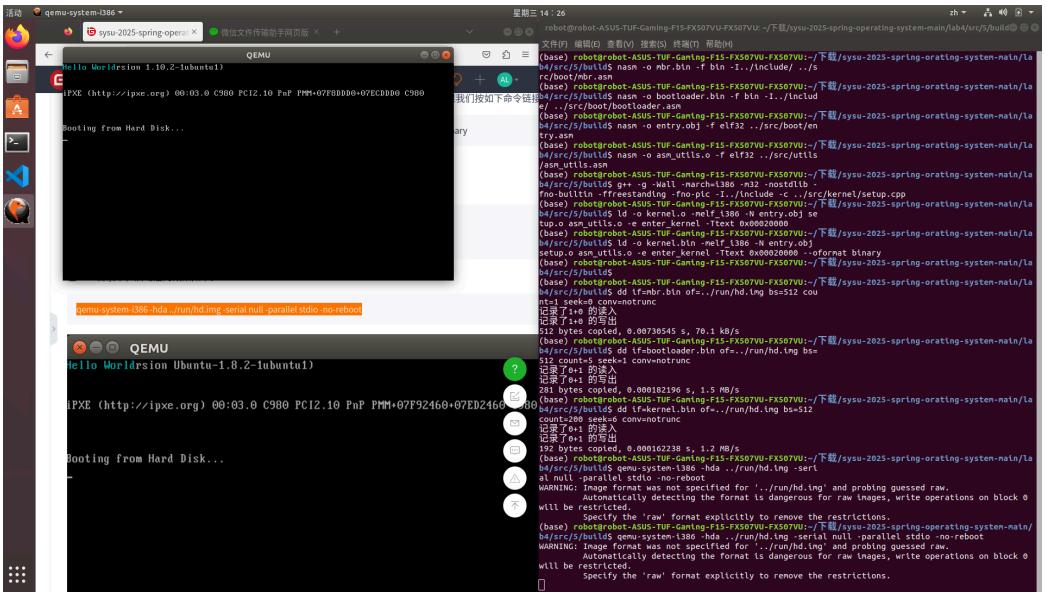
- 目标(target)：要生成的文件
- 依赖(dependency)：生成目标所需的文件
- 命令(command)：生成目标的具体指令，必须以Tab开头

编译步骤解析：

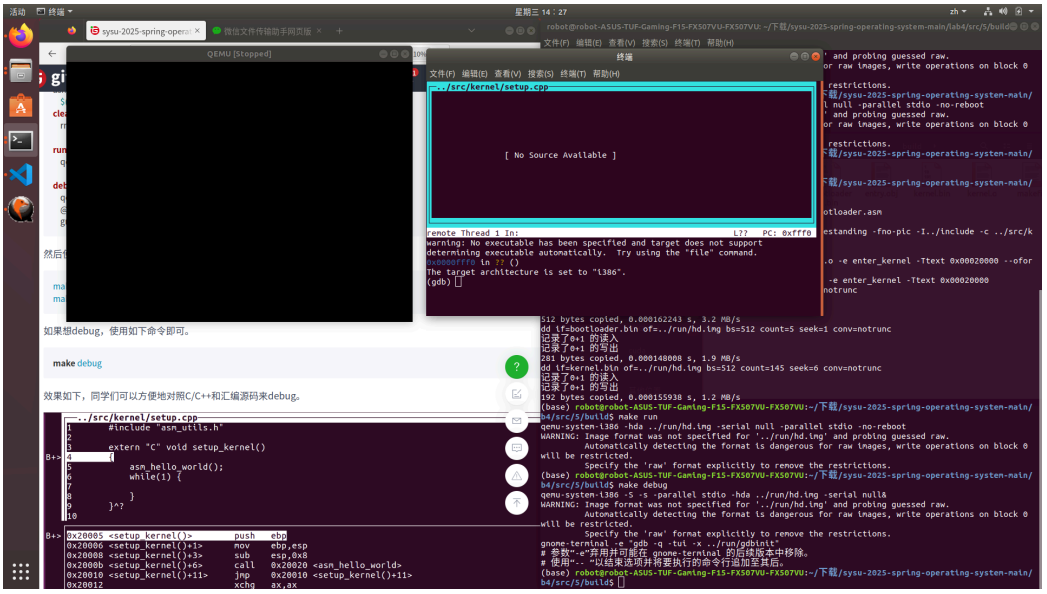
1. 将各源文件编译为目标文件(.o)
2. 将目标文件链接为可执行文件
3. 提供clean规则清理编译产

Assignment 2 使用C/C++来编写内核

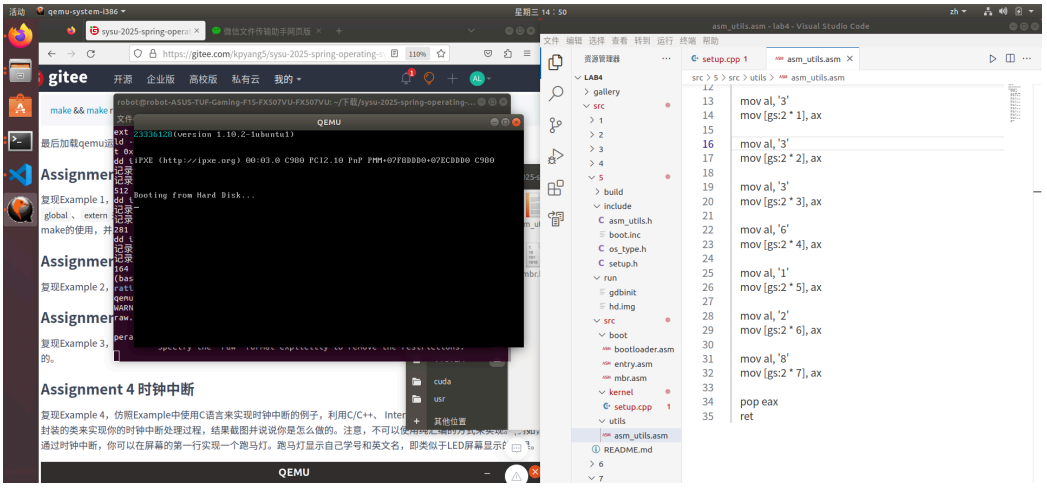
复现example2



make debug



输出学号



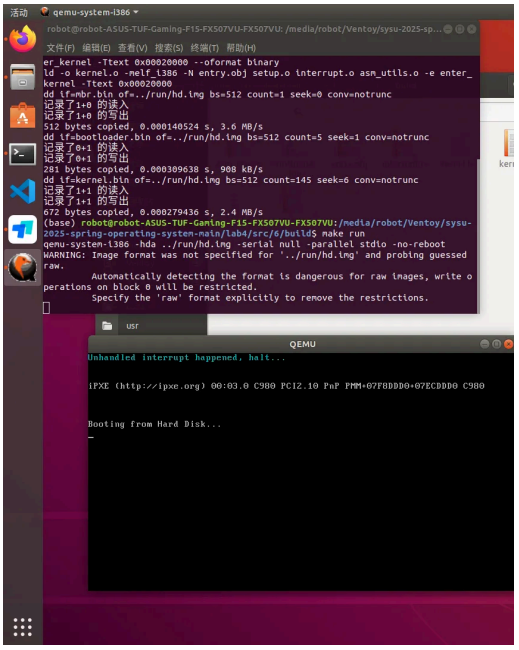
实现方法：

将中断处理函数中的输出字符逐一替换为自己的学号即可

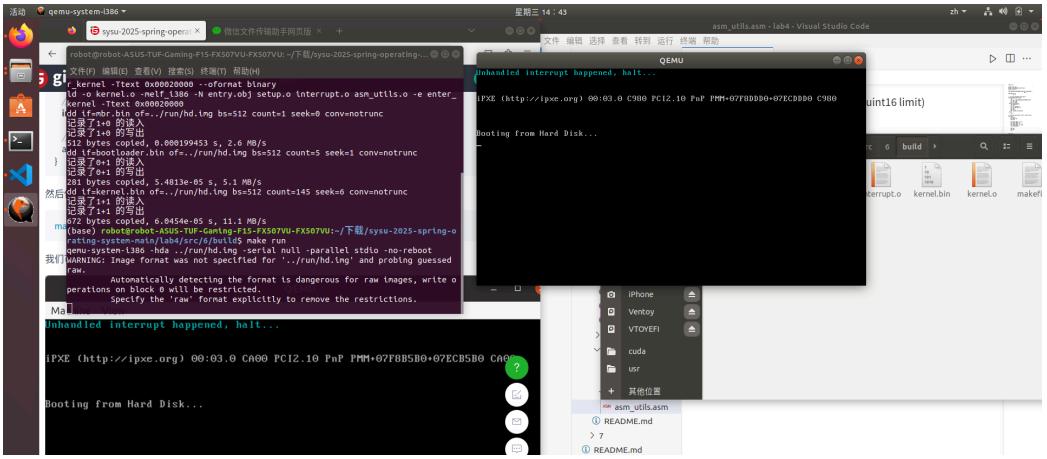
- 修改asm_hello_world函数或重新编写一个汇编函数
  - 在显存地址写入学号的ASCII码和颜色信息
  - 需要理解显存映射方式：每个字符占用2字节(ASCII码+颜色属性)

Assignment 3 中断的处理

复现Example 3



修改中断处理函数



自定义中断处理函数实现

1.***函数功能：**

- 替换默认的 `asm_unhandled_interrupt` 为 `my_interrupt_handler`。

- 在中断发生时输出自定义信息。

2. **触发测试：**

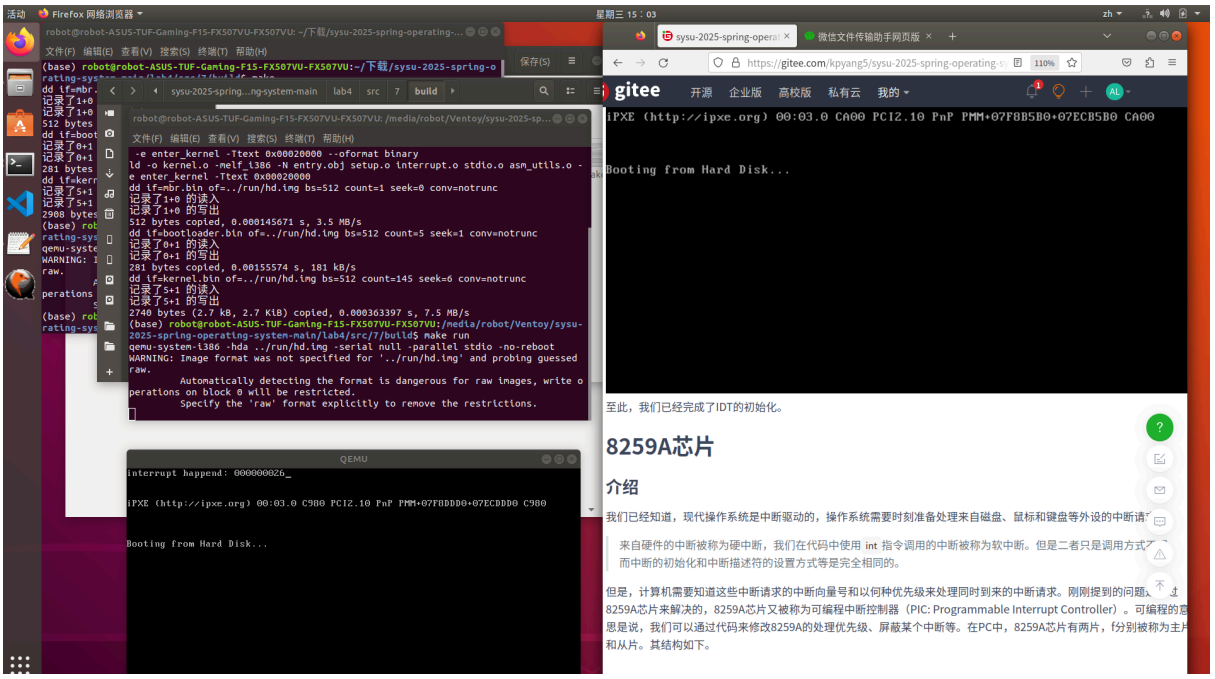
- 通过 `int $0x00` 指令主动触发中断，验证处理逻辑。

3. **结果验证：**

- 屏幕正确显示自定义中断信息，证明中断处理函数已生效。

Assignment 4 时钟中断

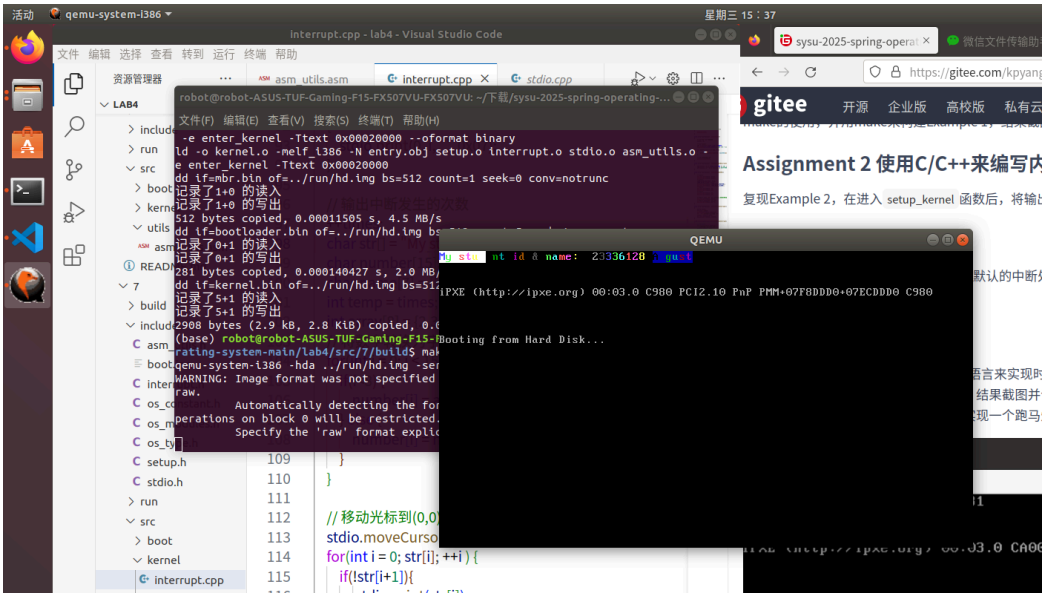
复现Example 4



时钟中断开启流程：

- 初始化中断描述符表(IDT)。
- 初始化8259A芯片。
- 设置时钟中断处理函数。
- 打开时钟中断。
- 开启全局中断(sti指令)。

跑马灯程序



3. 关键代码

输出学号

```
[bits 32]

global asm_hello_world

asm_hello_world:
    push eax
    xor eax, eax

    mov ah, 0x03 ;青色
    mov al, '2'
    mov [gs:2 * 0], ax

    mov al, '3'
    mov [gs:2 * 1], ax

    mov al, '3'
    mov [gs:2 * 2], ax

    mov al, '3'
    mov [gs:2 * 3], ax
```

```
    mov al, '6'
    mov [gs:2 * 4], ax

    mov al, '1'
    mov [gs:2 * 5], ax

    mov al, '2'
    mov [gs:2 * 6], ax

    mov al, '8'
    mov [gs:2 * 7], ax

    pop eax
    ret
```

中断处理函数

```
; 自定义中断处理函数
global my_interrupt_handler
my_interrupt_handler:
    cli
    mov esi, MY_INTERRUPT_MESSAGE ；自定义中断信息
    xor ebx, ebx
    mov ah, 0x0F ；白色字体
.output_loop:
    cmp byte [esi], 0
    je .end
    mov al, byte [esi]
    mov word [gs:bx], ax
    inc esi
    add ebx, 2
    jmp .output_loop
.end:
    iret
```

```
void InterruptManager::initialize()
{
    // 初始化IDT
    IDT = (uint32 *)IDT_START_ADDRESS;
    asm_lidt(IDT_START_ADDRESS, 256 * 8 - 1);

    // for (uint i = 0; i < 256; ++i)
    // {
    //     setInterruptDescriptor(i, (uint32)asm_unhandled_interrupt, 0);
    // }

    // 自定义的异常处理
    for (uint i = 0; i < 256; ++i)
    {
        setInterruptDescriptor(i, (uint32)my_interrupt_handler, 0);
    }
}
```

```
extern "C" void setup_kernel()
{
    // 中断处理部件
    interruptManager.initialize();

    // 尝试触发除0错误
    int a = 1 / 0;
    asm volatile("int $0x00");  // 触发中断 0x00（除零异常）
    // 死循环
    asm_halt();
}
```

跑马灯

```
extern "C" void c_time_interrupt_handler()
{
    // 清空屏幕
    for (int i = 0; i < 80; ++i)
```

```
    {
        stdio.print(0, i, ' ', 0x07);
    }

    // 输出中断发生的次数
    ++times;
    char str[] = "My student id & name:  ";
    char number[15];
    char name[7] = {' ','A', 'u', 'g','u','s','t'};
    int temp = times;
    int array[8] = {2,3,3,3,6,1,2,8};
    // 将数字转换为字符串表示
    for(int i = 0; i < 15; ++i ) {
        if(i<8) {
            number[i] = array[i] + '0';
        } else {
            number[i] = name[i-8];
        }
    }

    // 移动光标到(0,0)输出字符
    stdio.moveCursor(0);
    for(int i = 0; str[i]; ++i ) {
        if(!str[i+1]){
             stdio.print(str[i]);
             continue;
        }
        stdio.print(str[i],(i-times)%0xff);
    }

    // 输出学号
    for( int i = 0; i < 15; ++i ) {
        stdio.print(number[i], (i+times)%0xff);
    }
}
```

4. 实验结果

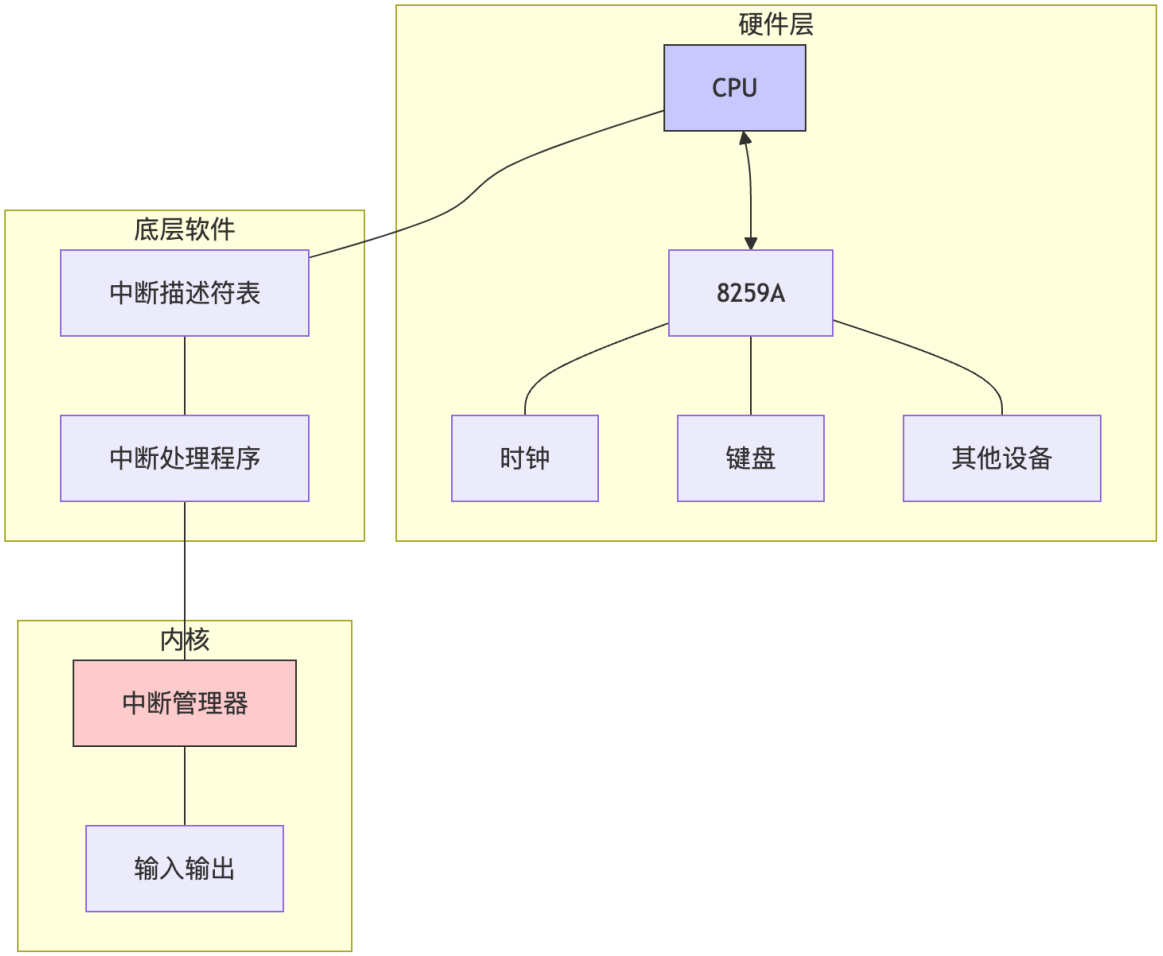
实验结果同第二部分，已经展示，此处不重复

5. 总结

通过本次实验，我深入学习了C/C++与汇编混合编程的基本原理，掌握了保护模式下中断描述符表（IDT）的设置方法，以及8259A芯片的初始化和编程技巧。同时，我实现了时钟中断处理函数，并对保护模式下的中断处理机制有了全面的理解。实验中，我不仅学习了中断的理论知识，还通过实践掌握了中断现场的保存与恢复技术，以及如何综合运用C/C++和汇编语言实现功能。

在实验过程中，我遇到了一些难点，例如理解C函数调用规则与寄存器的使用、保护模式中断处理机制的全过程，以及如何在混合编程中协调C/C++和汇编代码的协作。通过反复调试和查阅资料，我逐渐克服了这些困难，进一步加深了对操作系统底层机制的理解。

实验结束后，我对C代码从源文件到可执行文件的转换过程有了更清晰的认识，同时也掌握了C/C++与汇编混合编程的技术要点。实验让我深刻体会到理论知识与实践结合的重要性，不仅提升了我的编程能力，也让我对操作系统的底层实现有了更直观的感受。


硬件层
CPU
8259A
时钟
键盘
其他设备
底层软件
中断描述符表
中断处理程序
内核
中断管理器
输入输出